COURSE: COMPUTER ARCHITECTURE

COURSE CODE: 25AIM34

SEM/SEC: III E

NAME: VEDIKA RAJ R

USN: 1NH25AI156-T

Q1) Develop a Python program to perform signed binary arithmetic using the 8-bit 2's complement representation. Use the program to verify the results of the following arithmetic expressions:

a) (25 + (+18))
b) ((-25) + (-18))
For each expression, display the operands in 8-bit 2's complement binary format, perform the binary arithmetic operation, and display the final result in both binary and decimal formats. Verify the program output against the results obtained through manual binary calculations and identify any arithmetic overflow.

SOLUTION:

Signed Binary Arithmetic Using 8-bit 2's Complement

Given: Perform the following arithmetic expressions using 8-bit 2's complement representation.

- a) (25 + (+18))
- b) ((-25) + (-18))

Range of 8-bit 2's complement:

-128 to +127

a) (25 + (+18))

Step 1: Convert 25 into 8-bit binary

25 = 00011001

Step 2: Convert +18 into 8-bit binary

18 = 00010010

Since both numbers are positive, their MSB (Most Significant Bit) is 0.

Step 3: Perform binary addition

```
  00011001    (25)
+ 00010010    (+18)
------------
  00101011
```

Step 4: Convert the result into decimal

00101011 = 32+8+2+1
=43
No overflow

Final Answer:

- Binary result: `00101011`
- Decimal result: `+43`
- Overflow: No

b) ((-25) + (-18))

Step 1: Convert -25 into 8-bit 2's complement

First, convert +25 into binary:

25 = 00011001

Take 1's complement (invert all bits):

11100110

Add 1 to obtain 2's complement:

```
  11100110
+ 00000001
----------
  11100111
```

Therefore,

-25 = 11100111

Step 2: Convert -18 into 8-bit 2's complement

First, convert +18 into binary:

18 = 00010010

Take 1's complement:

11101101

Add 1:

```
  11101101
+ 00000001
----------
  11101110
```

Therefore,

-18 = 11101110

Step 3: Perform binary addition

```
  11100111    (-25)
+ 11101110    (-18)
------------
1 11010101
```

Discard the extra carry bit because we are using 8-bit representation.

Therefore,

Result = 11010101

Step 4: Convert the result into decimal

Since the MSB is 1, the result is negative.

To find its magnitude, take 2's complement.

Invert the bits:

11010101 = 00101010

Add 1:

```
  00101010
+ 00000001
----------
  00101011
```

00101011 = 43

Therefore,

11010101 = -43

No overflow

Final Answer:

- Binary result: `11010101`
- Decimal result: `-43`
- Overflow: No

PROGRAM:

main.py

```python
def to_8bit_binary(n):
    return format(n & 0xFF, '08b')


def signed_binary_add(a, b):
    binary_a = to_8bit_binary(a)
    binary_b = to_8bit_binary(b)

    result_unsigned = (a & 0xFF) + (b & 0xFF)
    result_8bit = result_unsigned & 0xFF

    # Convert the 8-bit result to signed decimal
    if result_8bit >= 128:
        result_decimal = result_8bit - 256
    else:
        result_decimal = result_8bit

    overflow = not (-128 <= a + b <= 127)

    print("First operand:", a, "=", binary_a)
    print("Second operand:", b, "=", binary_b)
    print("Binary addition:", binary_a, "+", binary_b)
    print("8-bit result:", format(result_8bit, '08b'))
    print("Decimal result:", result_decimal)
    print("Arithmetic overflow:", "Yes" if overflow else "No")
    print("-" * 40)


print("Expression (a): 25 + (+18)")
signed_binary_add(25, 18)
```

Output

```
Expression (a): 25 + (+18)
First operand: 25 = 00011001
Second operand: 18 = 00010010
Binary addition: 00011001 + 00010010
8-bit result: 00101011
Decimal result: 43
Arithmetic overflow: No
----------------------------------------
Expression (b): (-25) + (-18)
First operand: -25 = 11100111
Second operand: -18 = 11101110
Binary addition: 11100111 + 11101110
8-bit result: 11010101
Decimal result: -43
Arithmetic overflow: No
----------------------------------------
```

main.py

```python
def signed_binary_add(a, b):
    binary_a = to_8bit_binary(a)
    binary_b = to_8bit_binary(b)

    result_unsigned = (a & 0xFF) + (b & 0xFF)
    result_8bit = result_unsigned & 0xFF

    # Convert the 8-bit result to signed decimal
    if result_8bit >= 128:
        result_decimal = result_8bit - 256
    else:
        result_decimal = result_8bit

    overflow = not (-128 <= a + b <= 127)

    print("First operand:", a, "=", binary_a)
    print("Second operand:", b, "=", binary_b)
    print("Binary addition:", binary_a, "+", binary_b)
    print("8-bit result:", format(result_8bit, '08b'))
    print("Decimal result:", result_decimal)
    print("Arithmetic overflow:", "Yes" if overflow else "No")
    print("-" * 40)


print("Expression (a): 25 + (+18)")
signed_binary_add(25, 18)

print("Expression (b): (-25) + (-18)")
signed_binary_add(-25, -18)
```

Output

```
Expression (a): 25 + (+18)
First operand: 25 = 00011001
Second operand: 18 = 00010010
Binary addition: 00011001 + 00010010
8-bit result: 00101011
Decimal result: 43
Arithmetic overflow: No
----------------------------------------
Expression (b): (-25) + (-18)
First operand: -25 = 11100111
Second operand: -18 = 11101110
Binary addition: 11100111 + 11101110
8-bit result: 11010101
Decimal result: -43
Arithmetic overflow: No
----------------------------------------
```

Q2) Write a Python program to perform and verify the results of the following shift operations on 8-bit binary numbers:

a) Logical left shift of `10110110` by 2 positions.

b) Logical right shift of `11010100` by 2 positions.

For each operation, display the original 8-bit binary number, demonstrate the shifting process step by step, and display the final 8-bit binary result. Verify the program output against the results obtained through manual calculations. Ensure that the output remains in 8-bit format.

SOLUTION:

Logical Shift Operations on 8-bit Binary Numbers

Given: Perform the following logical shift operations by 2 positions.

a) Logical Left Shift of `10110110` by 2 positions

Step 1: Original binary number

10110110

Step 2: Shift left by 1 position

- Shift all bits one position to the left.
- Discard the leftmost bit.
- Add 0 at the rightmost position.

```
Original:       10110110
After 1 shift:  01101100
```

Step 3: Shift left by 2 positions

Shift one more position to the left.

```
After 1 shift:  01101100
After 2 shifts: 11011000
```

Final Answer
11011000

b) Logical Right Shift of `11010100` by 2 positions

Step 1: Original binary number

11010100

Step 2: Shift right by 1 position

- Shift all bits one position to the right.

- Discard the rightmost bit.
- Add 0 at the leftmost position.

```
Original:       11010100
After 1 shift:  01101010
```

Step 3: Shift right by 2 positions

Shift one more position to the right.

```
After 1 shift:  01101010
After 2 shifts: 00110101
```

Final Answer
00110101

PROGRAM:

